

Practica 13

Programacion orientada a objetos

Jesus collazo

Cesar zaid reynoso martinez

Introducción

¿Qué es una clase abstracta?

Una clase abstracta es una clase base que no puede crear objetos directamente. Se utiliza como modelo para que otras clases hereden sus atributos y métodos. Además, puede contener métodos virtuales puros que obligan a las clases derivadas a implementar ciertos comportamientos.

En este sistema, la clase Empleado es abstracta porque representa una estructura general para todos los empleados de la empresa.

¿Qué es una interfaz?

Una interfaz es una clase que únicamente contiene métodos virtuales puros. Su función es obligar a otras clases a implementar ciertos métodos específicos.

En esta práctica se utilizó la interfaz IResponsable, la cual obliga a todas las clases derivadas a implementar el método generarReporte().

¿Cuál es la diferencia entre ambas?

La diferencia principal es que:

La clase abstracta puede tener atributos, métodos normales y métodos virtuales.

La interfaz solamente contiene métodos virtuales puros y no almacena datos.

La clase abstracta sirve como base general y la interfaz define comportamientos obligatorios.

¿Por qué son importantes en POO?

Son importantes porque permiten crear programas más organizados, reutilizables y flexibles. También facilitan el polimorfismo y permiten agregar nuevas clases sin modificar gran parte del código existente

Desarrollo

Explicación del sistema

El sistema desarrollado permite administrar diferentes tipos de empleados dentro de una empresa tecnológica.

Se implementaron tres tipos de empleados:

1. Programador
2. Administrador de Red
3. Soporte Técnico

Cada empleado puede:

- Mostrar información
- Trabajar según su función
- Generar reportes
- Realizar actividades específicas

El programa utiliza herencia, clases abstractas, interfaces y polimorfismo para manejar todos los empleados de forma general.

1. Diseño de clases
 - Clase abstracta: Empleado

Contiene:

Atributos:

- nombre
- edad
- salario
- bono mensual

Métodos:

- mostrarInformacion()
- trabajar()
- realizarActividad()
- calcularSalarioTotal()
- El método trabajar() fue declarado virtual puro.

```

#ifndef EMPLEADO_H
#define EMPLEADO_H

#include <iostream>
#include <vector>
using namespace std;

// Interface
class IResponsable {
public:
    virtual void generarReporte() = 0;
};

// Clase abstracta
class Empleado {
protected:
    string nombre;
    int edad;
    float salario;
    float bono;
public:
    Empleado(string n, int e, float s, float b) {
        nombre = n;
        edad = e;
        salario = s;
        bono = b;
    }

    virtual void trabajar() = 0;

    virtual void mostrarInformacion() {
        cout << "\nNombre: " << nombre;
        cout << "\nEdad: " << edad;
        cout << "\nSalario: $" << salario;
        cout << "\nBono: $" << bono;
        cout << "\nSalario Total: $" << salario + bono << endl;
    }
};

```

Empleado.h

Este módulo contiene la base principal del sistema. Aquí se creó la clase abstracta Empleado y la interfaz IResponsable.

La clase Empleado guarda la información general que todos los empleados tienen en común, como el nombre, la edad, el salario y el bono. También incluye métodos para mostrar información y realizar actividades. Además, el método trabajar() se declaró como virtual puro para obligar a que cada tipo de empleado lo implemente de manera diferente.

En este mismo módulo también se encuentra la interfaz IResponsable, la cual obliga a las clases derivadas a tener un método llamado generarReporte(). Gracias a esto, todos los empleados pueden generar reportes según su función.

```

#include "Empleado.h"

void menu(vector<Empleado*>& empleados) {
    int opcion;

    do {
        cout << "\n==== MENU =====";
        cout << "\n1. Agregar Programador";
        cout << "\n2. Agregar Administrador de Red";
        cout << "\n3. Agregar Soporte Tecnico";
        cout << "\n4. Mostrar empleados";
        cout << "\n5. Ejecutar trabajos";
        cout << "\n6. Generar reportes";
        cout << "\n7. Salir";
        cout << "\nOpcion: ";
        cin >> opcion;

        string nombre;
        int edad;
        float salario, bono;

        switch(opcion) {

            case 1:
                cout << "Nombre: ";
                cin >> nombre;
                cout << "Edad: ";
                cin >> edad;
                cout << "Salario: ";
                cin >> salario;
                cout << "Bono: ";
                cin >> bono;

```

Funciones.h

En este módulo se crean las clases derivadas:

- Programador
- AdministradorRed
- SoporteTecnico

Cada una hereda de la clase Empleado e implementa la interfaz IResponsable.

Aquí se definen los comportamientos específicos de cada empleado. Por ejemplo:

El programador desarrolla código.

El administrador de red monitorea servidores.

El soporte técnico atiende incidencias.

También cada clase implementa su propia forma de generar reportes. Este módulo es importante porque aplica la herencia y el polimorfismo, permitiendo que cada objeto responda de forma distinta aunque todos se manejen como empleados generales.

Main.cpp

Este módulo contiene el programa principal y es donde se ejecuta todo el sistema.

Aquí se crea un vector de punteros tipo Empleado* para almacenar diferentes tipos de empleados. Después, el programa recorre el vector utilizando polimorfismo para mostrar información, ejecutar actividades y generar reportes.

El main.cpp se encarga de conectar todos los módulos y demostrar cómo funcionan juntos las clases abstractas, las interfaces y el polimorfismo.

Además, en este módulo se observa claramente cómo un mismo método puede comportarse de manera distinta dependiendo del tipo de empleado que lo ejecute.

EXPLICACION TECNICA

¿Cómo se implementó la clase abstracta?

La clase abstracta se implementó creando la clase Empleado y declarando el método virtual puro

¿Cómo se implementó la interfaz?

La interfaz IResponsable se creó usando únicamente métodos virtuales puros

¿Qué ventajas tiene usar interfaces?

Las interfaces permiten:

- Obligar a implementar ciertos métodos
- Mantener el código organizado
- Facilitar la reutilización
- Agregar nuevas clases fácilmente

¿Cómo ayuda el polimorfismo en este sistema?

El polimorfismo permite manejar distintos tipos de empleados usando un mismo tipo base (Empleado*).

Gracias a esto, el programa puede recorrer todos los empleados de forma general y cada objeto ejecuta comportamientos distintos automáticamente.

CONCLUSIONES

En esta práctica aprendí cómo funcionan las clases abstractas, las interfaces y el polimorfismo dentro de la Programación Orientada a Objetos en C++. Comprendí cómo una clase abstracta puede servir como base para diferentes tipos de objetos y cómo las interfaces ayudan a obligar ciertos comportamientos en las clases derivadas.

Una de las principales dificultades fue entender cómo implementar correctamente la herencia múltiple utilizando interfaces y cómo utilizar punteros para aplicar polimorfismo. También fue un reto organizar el programa en módulos para que el código estuviera más limpio y ordenado.

Considero que este tipo de sistemas puede aplicarse fácilmente en empresas reales porque permite administrar diferentes tipos de empleados de manera flexible, reutilizable y fácil de mantener.